ສະບາຍດີ!

SABAIDEE! · STUDENT TALK

A Cabin That Keeps Itself Warm

An autonomous heating system built with a Raspberry Pi, Bluetooth thermometers and a smart plug

RYU Cheol · ETRI · 6 November 2026

THE PROBLEM

An empty cabin in winter

Pipes can freeze

Nobody is there for months. Water pipes crack when the cabin gets too cold.

Warm on arrival

On a visit the cabin should already be comfortable, without heating all week.

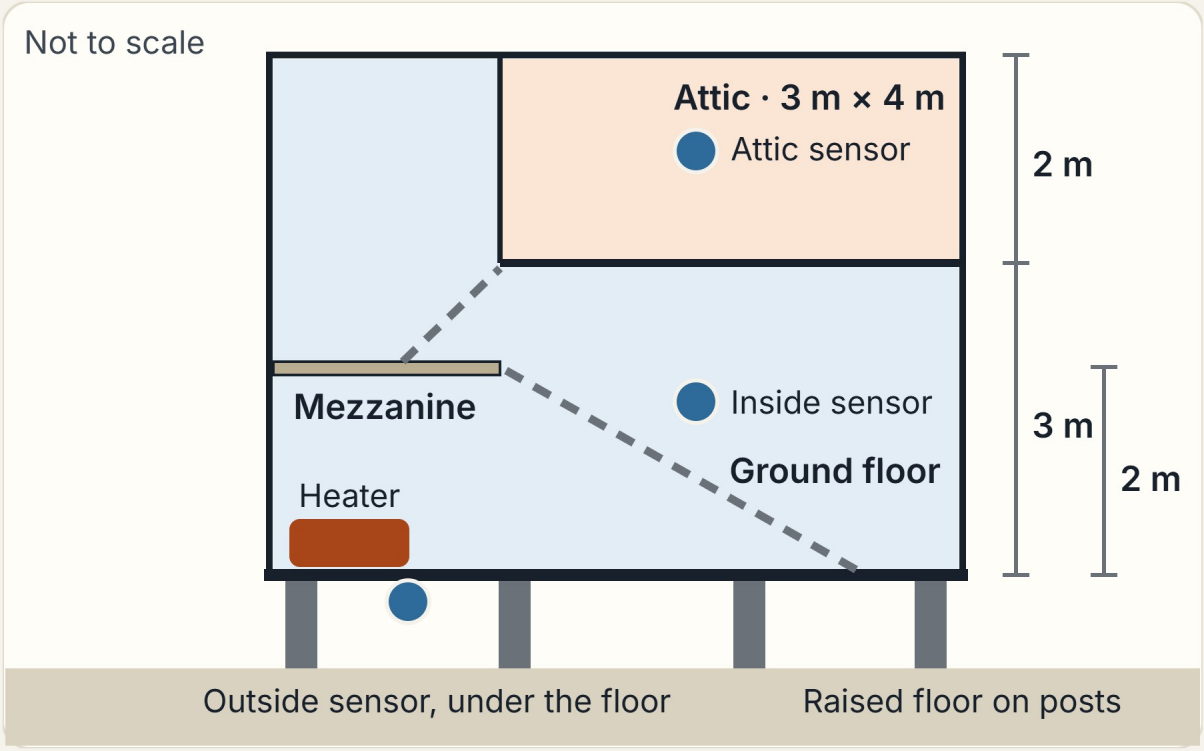
No steady internet

A cloud service fails when the connection drops. The decision must be made on site.

Goal: protect the cabin automatically, and never depend on the internet.

THE CABIN

Three levels, three sensors

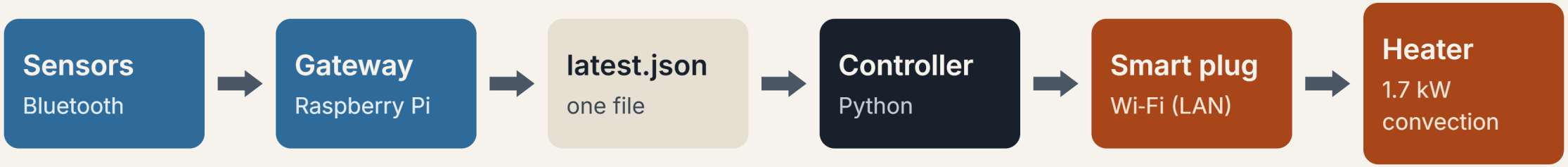




Ground floor 3 m high · attic 2 m high · mezzanine 2 m above the floor, open to the ceiling · raised floor on posts

BIG PICTURE

Sense, think, act



Sense Think Act

Two independent programs

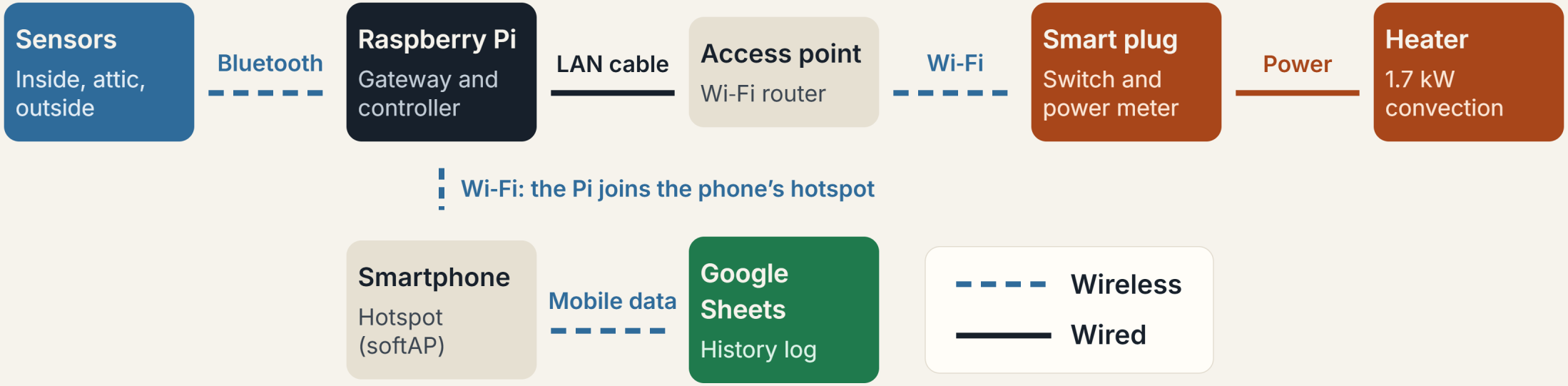
The gateway and the controller only share one file. If one crashes, the other keeps running.

Local first

Control needs no internet. Google Sheets logging is a bonus when a connection exists.

NETWORK

How the devices talk



Control stays local

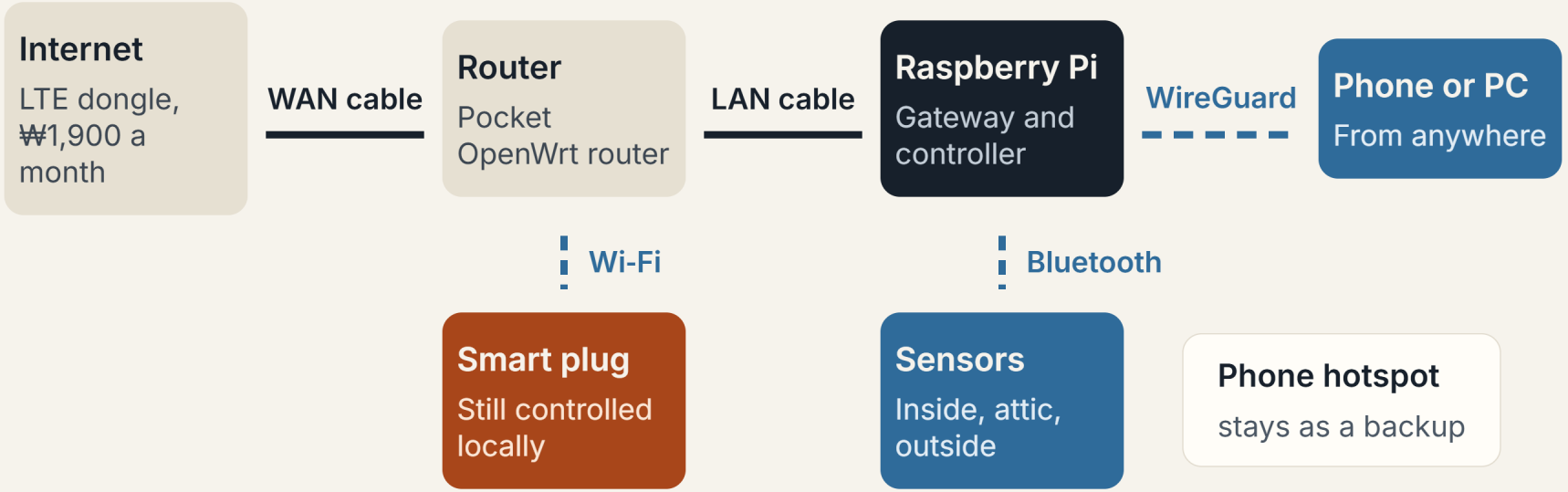
Pi, access point and plug are all inside the cabin network. The heater needs no internet.

History goes out when possible

Without the hotspot the Pi saves readings locally and uploads them later.

NEXT STEP

Going online: LTE this month, fibre next



Always uploading

Readings and frost alerts reach the Sheet even when nobody is there. Fibre (100 Mbps) is available at the cabin and will take over as the main line.

Log in from afar

Remote login with no open ports on the router.

Presence, logged by itself

Arrival = the phone's hotspot appears. Departure = the frost command. Written to the Sheet automatically since 8 Nov.

SENSE

Cheap sensors, quietly listening

Xiaomi LYWSD03MMC

A small Bluetooth thermometer and hygrometer, very low cost.

Passive listening

The Pi never connects; it only hears the broadcasts. With the open **pvvx** firmware (since 7–8 Nov) a new reading arrives **every minute** instead of every 12, and the radio is on less often, so batteries last longer.

Encrypted messages

Each sensor has its own key. The Pi decrypts every broadcast to read the values. The new firmware kept Xiaomi's format, so nothing changed on the Pi.

Store and forward

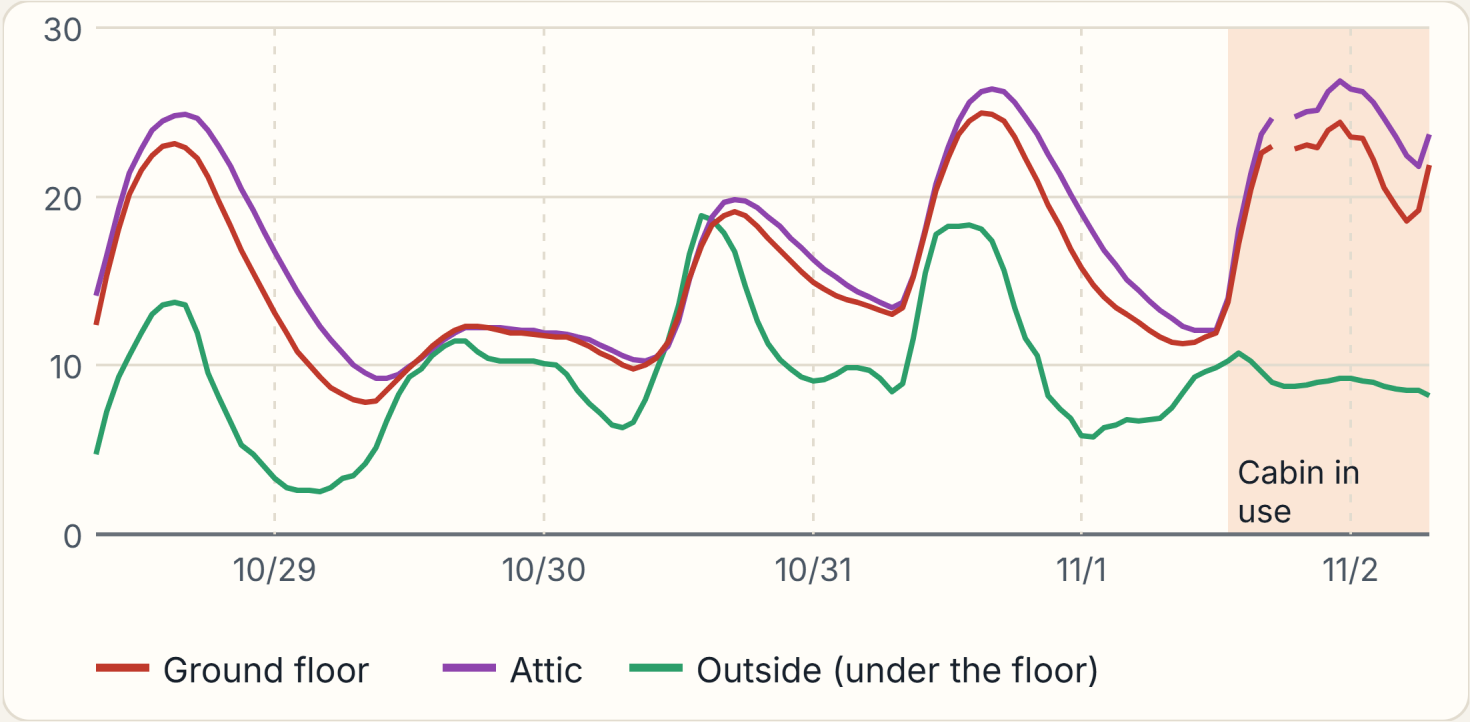
Readings are saved on the Pi and uploaded to Google Sheets when internet is available.

Sensor	Used for
Inside	Frost and comfort control
Outside	Outdoor temperature for preheating
Attic	Sleeping mode and backup sensor

Three sensors, three places, three jobs.

DATA

Every reading lands in a Google Sheet



Hourly temperature, °C · real data

Until the owner arrives (1 Nov, 13:00) the rooms simply follow the weather. Afterwards the heater lifts the ground floor to about 23 °C.

Now	°C	% RH
Attic	25.5	44.2
Outside	8.9	90.8
Ground floor	23.1	45.5

Snapshot from 2 Nov, 08:57. The Pi uploads whenever it has internet.

ACT

A smart plug switches a 0.7–1.8 kW heater

3 steps

720, 1,102 or 1,763 W, chosen by a switch on the heater and measured by the plug.

LAN

Commands travel Pi, router, plug. No cloud in the control loop.

3 in 1

Switch, power meter and energy counter in one device.

Why the power meter matters: if the plug is ON but the heater draws almost nothing, the system knows the heater has failed.

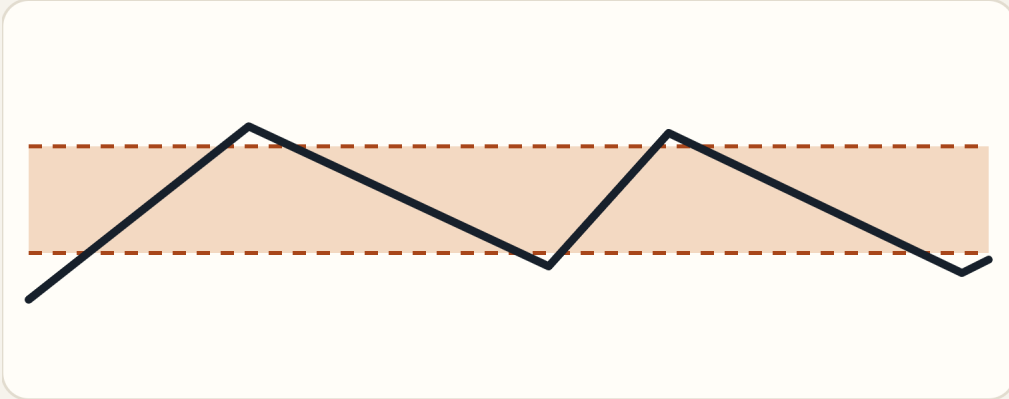
Less is smoother: at the 720 W step the overshoot measured on 8 Nov was 0.0 °C. Matching the heater to the heat loss beats clever timing.

THINK

Three modes, one simple rule

- Below the ON value: heater ON
- Above the OFF value: heater OFF
- In between: keep the current state

Why a gap? Without it the relay would click on and off every minute.



Mode	When	Sensor	ON below / OFF above
Frost	Nobody is there	Inside	3 °C / 5 °C
Comfort	You have arrived	Inside	23 °C / 23.5 °C
Sleep	Sleeping in the attic	Attic	20 °C / 21 °C

PLAN AHEAD

Preheating with physics

The owner usually arrives for the weekend at a fixed time: **Friday 20:00**.

Start early by

$$\tau \times \ln\left(\frac{T_{eq} - T_{in}}{T_{eq} - T_{target}}\right) \times \text{margin}$$

$$T_{eq} = T_{out} + \frac{P}{UA}$$

τ = 9.16 h (how slowly the cabin cools) · UA = 47 W/K (heat lost per degree) · P = 1,740 W (heater power) · margin = 1.25

Example: inside 5 °C, target 23 °C

Outside	Start before arrival
8 °C	6.8 hours
0 °C	9.5 hours
−10 °C	10 hours (limit)

1. Estimate

Hours of heating needed, from the outside temperature.

2. Start early

Switch to comfort mode before Friday 20:00 arrival.

3. No show?

No confirmation by midnight: back to frost mode.

SAFETY FIRST

Assume that things will fail

Stale sensor

No fresh reading for 45 minutes? In frost mode the heater turns ON.

Two thermometers

Ground floor and attic back each other up if one sensor dies.

Dead-man timer

While OFF, the plug's own countdown is renewed every 5 minutes. If the Pi dies, it expires and the plug turns ON.

Time limits

No new command for 36 hours? Comfort and sleep modes fall back to frost mode.

Fault detection

Plug ON but under 100 W for 5 minutes? The heater is probably broken. Log it.

Self-test

Before leaving, one command pulses the plug and checks the power: PASS or FAIL.

PROVE IT

Tests, simulation, and a real measurement

57 automated tests

The decision logic is pure functions, so it runs on a laptop. No heater needed.

A real bug, found by a test

The “no-show” check was inside a Friday-only block but waited for Saturday. It could never run. Writing the test exposed it.

Measured, 8 Nov: overshoot per cycle

0.4 → 0.1 °C

Same controller, same cabin: sensor reporting every 12 min (stock) vs every 1 min (new firmware). The simulator predicted it; the cabin confirmed it.

Simulated room with assumed parameters. People notice changes of about 0.4 °C (Battistel et al., 2023).

FIRMWARE UPDATE · 7–8 NOVEMBER

Teaching the sensors to speak every minute

The problem we found

The stock firmware broadcast every 2 seconds, but the **value** only changed every 12 minutes. The controller was always acting on old news.

What we did

Flashed the open-source **pvvx** firmware over Bluetooth from a laptop browser, no cable. Kept Xiaomi's encrypted message format, so the gateway code did not change. New settings: measure every 20 s, broadcast every 5 s.

Measured in the cabin	Stock	pvvx
New reading every	12 min	1 min
Overshoot past the OFF value	+0.4 °C	+0.1 °C
Temperature swing per cycle	1.1 °C	0.7 °C
Radio bursts per minute	29	12
Battery shown	"100 %"	67–96 % real

Same cabin, same rule (23 / 23.5 °C), same heater. The radio is the battery cost, not the measurement, so reporting 12× more often still uses *less* power.

Lesson: the control rule never changed. The sensor's latency, not its accuracy, was the whole problem.

WEATHER SERVICE

Using the public forecast, simply first

The feed

- Korea Meteorological Administration open API, free key
- Issued 8 times a day, 72 hours ahead, for a 5 km grid cell
- Hourly: temperature, sky, rain, humidity, wind
- The Pi fetches it every hour and keeps every issue

1 · Simple use, running today

One number from the forecast, the **minimum temperature ahead**, plus one line of physics: the warmest the room can get is **T_out + P / UA**.

- Someone home, next 12 h: can the current heater step hold the set-point? If not, say so.
- Empty cabin, next 48 h: cold snap too deep for this step? Warn before freezing.
- Leaving on Sunday: will Friday's preheat ever reach 23 °C at this step?

Real example · 8 Nov, 15:14, spoken aloud in the cabin

"Forecast low in the next 12 hours is 6 °C. At step 1 (702 W) the room can only hold 21 °C. Switch to step 2 or higher."

6 + 702 / 47 ≈ 21 °C, below the 23 °C target. The controller knew the step from the plug's power reading one minute after switching on.

2 · Full use, next slide

Feed the whole 72-hour forecast, cloud cover included, into a learned heat model and compute *when* to start, not just *whether* it can.

NEXT · LEARN

Let the cabin learn its own physics

Two heat balances, six unknowns

$$C_1 \cdot dT_1/dt = P \cdot u - UA_1(T_1 - T_out) - G(T_1 - T_2) + a \cdot Sun$$

$$C_2 \cdot dT_2/dt = G(T_1 - T_2) - UA_2(T_2 - T_out)$$

T_1 ground floor, T_2 attic · C heat capacity · UA heat loss · G coupling between floors · a effective window area · u heater on/off, P measured watts

Data, logged since 8 Nov

- Indoor temperatures and plug power, every minute
- Weather-service observations: wind, cloud, sunshine, radiation
- Every forecast issued, to score it later
- Arrivals and departures, written automatically

Why it matters

The heater adds only ~0.7 kW net. A sunny morning adds 1.4 kW through the east window. Weather decides the start time.

Fit, then trust

First fit after 2 weeks, validation after 4. Held-out days must predict within 0.3 °C an hour ahead.

Then use it

“Be 23 °C at 07:00”: start 2.5 h early on a cloudy day, 1.5 h on a sunny one. Let the sun finish the job.

YOUR TURN

What would you build in Laos?

Cooling

Fans or air conditioners that run only when a room is too hot.

Greenhouse

Keep temperature and humidity in range, even when the network is down.

Storage rooms

Protect rice, coffee or seeds from heat and damp.

Discussion: when your system fails, what is the safe state, ON or OFF?

Code and notes: github.com/ryucheol/sc-sensors

WHAT IT TAKES

Parts and budget



Raspberry Pi 4

$55–80 est.



microSD 32 GB

$5–10 est.



Thermometer × 3

₩17,544 · $12.5



Wi-Fi smart plug

₩4,650 · $3.3



Electric heater

₩59,900 · $42.8

Total of the five items

$119–149

Low: 55 + 5 + 12.5 + 3.3 + 42.8 ≈ 119

High: 80 + 10 + 12.5 + 3.3 + 42.8 ≈ 149

Questions?

Real prices: thermometer ₩5,848 each, plug ₩4,650 each, heater ₩59,900 (US$1 ≈ ₩1,400). Added Nov 2026: pvvx firmware (free), LTE router + dongle ≈ $45, SIM ₩1,900/month, two Shelly plugs with over-heat protection ≈ $21. Pi and microSD are estimates from AliExpress listings, Nov 2026; no power supply or shipping. Photos: Wikimedia Commons, Laserlicht and J. Halicki (CC BY-SA 4.0), plug and microSD (CC0); heater: product listing image.